

OS module

The os module in Python provides a way to interact with the operating system in a platform-independent manner.

Working with the File System

`os.listdir(path)` – List files and directories in the given path.

`os.getcwd()` – Get the current working directory.

`os.chdir(path)` – Change the current working directory.

`os.mkdir(path)` – Create a directory / nested directories.

`os.remove(path)` – Remove a file.

`os.rmdir(path)` – Remove a directory / nested directories.

`os.rename(src, dst)` – Rename a file or directory.

Renaming the file

The os module enables interaction with the operating system. The os module provides the functions that are involved in file processing operations like renaming, deleting, etc. It provides us the `rename()` method to rename the

specified file to a new name. The syntax to use the rename() method is given below.

Syntax:

```
rename(current-name, new-name)
```

The first argument is the current file name and the second argument is the modified name. We can change the file name bypassing these two arguments.

```
import os
#rename sample.txt to kochi.txt
os.rename("C:\\Users\\Username\\Desktop\\sample.txt", "C:\\Users\\Username\\Desktop\\data.txt")
print("file renamed")
```

Removing the file

The os module provides the remove() method which is used to remove the specified file. The syntax to use the remove() method is given below.

Syntax:

```
remove(file-name)
```

```
import os
#Deleting the file named file3.txt
os.remove("C:\\Users\\Username\\Desktop\\sample.txt")
print("File removed")
```

Creating the new directory

The mkdir() method is used to create the directories in the current working directory. The syntax to create the new directory is given below.

Syntax:

`mkdir(directory name)`

```
import os
#Creating a new directory with name Workspace
os.mkdir("C:\\Users\\Username\\Desktop\\Workspace")
print("folder created")
```

Get current working directory

This method returns the current working directory.

The syntax to use the `getcwd()` method is given below.

Syntax

`os.getcwd()`

```
import os
print(os.getcwd())
print("current directory")
```

output:

C:\DataScience\Workspace
current directory

Changing the current working directory

The `chdir()` method is used to change the current working directory to a specified directory.

The syntax to use the `chdir()` method is given below.

Syntax

```
chdir("new-directory")
```

```
import os
print("Current working directory")
print(os.getcwd())
# Changing current directory with the new directory
os.chdir("c:\\DataScience\\")
#It will display the current working directory
print(os.getcwd())
```

output:

```
Current working directory
c:\DataScience\Workspace
c:\DataScience
```

Deleting directory

The rmdir() method is used to delete the specified directory.

The syntax to use the rmdir() method is given below.

```
import os
#Delete the directory named Test
os.rmdir("c:\\DataScience\\Test")
```

Syntax

```
os.rmdir(directory name)
```

Exception Handling

An exception can be defined as an unusual condition in a program resulting in the interruption in the flow of the program. Whenever an exception occurs, the program stops the execution, and thus the further code is not executed.

Therefore, an exception is the run-time errors that are unable to handle to Python script. An exception is a Python object that represents an error

Python provides a way to handle the exception so that the code can be executed without any interruption. If we do not handle the exception, the interpreter doesn't execute all the code that exists after the exception. Python has many built-in exceptions that enable our program to run without interruption and give the output. These exceptions are given below:

Common Exceptions:

Python provides the number of built-in exceptions, but here we are describing the common standard exceptions. A list of common exceptions that can be thrown from a standard Python program is given below.

1. ZeroDivisionError: Occurs when a number is divided by zero.

eg: `result = 10 / 0` # ZeroDivisionError: division by zero

2. NameError: It occurs when a name is not found. It may be local or global.

eg: `print(x)` # NameError: name 'x' is not defined

3. IndentationError: If incorrect indentation is given.

eg: `def func():`
 `print("Hello")` # IndentationError: expected an indented block

4. IOError: It occurs when Input Output operation fails.

eg: with open("file.txt", "r") as f:

```
data = f.read() # IOError if there's a problem reading the file
```

5. EOFError: It occurs when the end of the file is reached, and yet operations are being performed.

eg:

```
while True:
```

```
    line = file.readline()
```

```
    print(line)
```

6. SyntaxError: Raised when the Python interpreter encounters a syntax issue in the code

eg: if x == 10

```
    print("Hello") # SyntaxError due to missing colon
```

7. ValueError: Raised when a function receives an argument of the right type but inappropriate value

eg: int("hello") # ValueError: invalid literal for int() with base 10: 'hello'

8. FileNotFoundError: Raised when trying to open a file that does not exist.

eg: open("non_existent_file.txt") # FileNotFoundError: [Errno 2] No such file or directory

9. *AttributeError*: Raised when an attribute reference or assignment fails (e.g., calling a method on an object that doesn't have that method).

eg: `"hello".some_method()` # *AttributeError*: 'str' object has no attribute 'some_method'

10: *KeyError*: Raised when trying to access a dictionary with a key that doesn't exist.

eg: `my_dict = {"a": 1}`
`print(my_dict["b"])` # *KeyError*: 'b'

Problem without handling exceptions

Exception is an abnormal condition that halts the execution of the program. Suppose we have two variables a and b, which take the input from the user and perform the division of these values. What if the user entered the zero as the denominator? It will interrupt the program execution and through a *ZeroDivision* exception.

```
#Example
a = int(input("Enter a:"))
b = int(input("Enter b:"))
c = a/b
print("a/b = %d" % c)
#other code:
print("Hi I am other part of the program")
```

output:

Enter a:5

Enter b:0

ZeroDivisionError

Traceback (most

recent call last)

~\AppData\Local\Temp\ipykernel_13052\361789826.py in

<module>

2 a = int(input("Enter a:"))

3 b = int(input("Enter b:"))

----> 4 c = a/b

5 print("a/b = %d" % c)

6 #other code:

ZeroDivisionError: division by zero

The above program is syntactically correct, but it through the error because of unusual input. That kind of programming may not be suitable or recommended for the projects because these projects are required uninterrupted execution. That's why an exception-handling plays an essential role in handling these unexpected exceptions.

Try-except statement

If the Python program contains suspicious code that may throw the exception, we must place that code in the try block. The try block must be followed with the except statement, which contains a block of code that will be executed if there is some exception in the try block.

Syntax

try:

 #block of code

except Exception1:

```
    #block of code

except Exception2:
    #block of code

#rest of the program
```

#Example

```
try:
    var1 = int(input("Enter var1:"))
    var2 = int(input("Enter var2:"))
    myvar = var1/var2
    print("result:",myvar)
    print("hi..")
except:
    print("Can't divide with zero")

print("outside exception")
```

output:

Enter a:5

Enter b:0

Can't divide with zero

outside exception

We can also use the `else` statement with the `try-except` statement in which, we can place the code which will be executed in the scenario if no exception occurs in the `try` block.

The syntax to use the `else` statement with the `try-except` statement is given below.

Syntax

`try:`

`#block of code`

`except Exception1:`

`#block of code`

`except Exception2:`

`#block of code`

`...`

`else:`

`#this code executes if no except block is executed`

`#Example`

`try:`

`var1 = int(input("Enter var1:"))`

`var2 = int(input("Enter var2:"))`

`myvar = var1/var2`

`print("var1/var2 = %d" % myvar)`

`# Using Exception with except statement. If we print(Exception) it will return exception class`

`except Exception:`

`print("can't divide by zero")`

`print(Exception)`

`else:`

`print("Hi I am else block")`

`print("outside exception")`

`output:`

`Enter var1:5`


```
Enter var2:0
can't divide by zero
<class 'Exception'>
outside exception
```

The except statement with no exception Python provides the flexibility not to specify the name of exception with the exception statement.

```
try:
    var1 = int(input("Enter var1:"))
    var2 = int(input("Enter var2:"))
    myvar = var1/var2
    print("var1/var2 = %d"%myvar)
except:
    print("can't divide by zero")
else:
    print("Hi I am else block")

print("outside exception")
```

output:

```
Enter var1:5
Enter var2:2
var1/var2 = 2
Hi I am else block
outside exception
```

except statement using with exception variable

We can use the exception variable with except statement. It is used by using the as keyword. this object will return the cause of the exception.

```
try:
    var1 = int(input("Enter a:"))
    var2 = int(input("Enter b:"))
    myvar = var1/var2
    print("var1/var2 = %d"%myvar)
    # Using exception object with the except statement
except Exception as e:
    print("can't divide by zero")
    print(e)
else:
    print("Hi I am else block")
print("outside exception")
```

output:

Enter var1:5

Enter var2:0

can't divide by zero

division by zero

outside exception

Points to remember

1. Python facilitates us to not specify the exception with the except statement.
2. We can declare multiple exceptions in the except statement since the try block may contain the statements which throw the different type of exceptions.
3. We can also specify an else block along with the try-except statement,

which will be executed if no exception is raised in the try block.

4. The statements that don't throw the exception should be placed inside the else block.

```
try:
    var1 = int(input("Enter var1:"))
    var2 = int(input("Enter var2:"))
    myvar = var1/var2
    print("var1/var2 = %d" % myvar)
    # multiple exception
except IOError as e1:
    print("IO Exception",e1)
except ArithmeticError as e2:
    print("Arithmetic Exception", e2)
else:
    print("Successfully Done")

print("outside exception")
```

output:

Enter var1:5

Enter var2:0

Arithmetic Exception division by zero

outside exception

Declaring Multiple Exceptions

The Python allows us to declare the multiple exceptions with the except clause. Declaring multiple exceptions is useful in the cases where a try block throws multiple exceptions. The syntax is given below.

Syntax

```
try:
```

```
#block of code
```

```
except (<Exception 1>,<Exception 2>,<Exception  
3>,...<Exception n>)
```

```
#block of code
```

```
else:
```

```
#block of code
```

```
try:
```

```
var1 = int(input("Enter var1:"))
```

```
var2 = int(input("Enter var2:"))
```

```
myvar = var1/var2
```

```
print("var1/var2 = %d"%myvar)
```

```
# multiple exception
```

```
except(ArithmeticError, IOError):
```

```
print("Arithmetic Exception")
```

```
else:
```

```
print("Successfully Done")
```

```
print("outside exception")
```

```
output:
```

```
Enter var1:5
```

```
Enter var2:0
```

```
Arithmetic Exception
```

```
outside exception
```

try...finally block

Python provides the optional finally statement, which is used with the try statement. It is executed no matter what exception occurs and used to release the external resource. The finally block provides a guarantee of the execution. We can use the finally block with the try block in which we can place the necessary code, which must be executed before the try statement throws an exception.

The syntax to use the finally block is given below.

Syntax

try:

block of code

this may throw an exception

finally:

block of code

this will always be executed

try:

```
var1 = int(input("Enter var1:"))
```

```
var2 = int(input("Enter var2:"))
```

```
myvar = var1/var2
```

```
print("var1/var2 = %d"%myvar)
```

```
# Using exception object with the except statement
```

except Exception as e:

```
print("can't divide by zero")
```

```
print(e)
```

else:

```
print("Hi I am else block")
```

finally:

```
print("inside finally")
```

```
print("outside exception")
```

output:

Enter var1:5

Enter var2:0

can't divide by zero

division by zero

inside finally

outside exception

Raising exceptions:

An exception can be raised forcefully by using the raise clause in Python. It is useful in that scenario where we need to raise an exception to stop the execution of the program.

The syntax to use the raise statement is given below.

Syntax

```
raise Exception_class(<value>)
```

Points to remember

1. To raise an exception, the raise statement is used. The exception class name follows it.
2. An exception can be provided with a value that can be given in the parenthesis.

3. To access the value "as" keyword is used. "e" is used as a reference variable which stores the value of the exception.
4. We can pass the value to an exception to specify the exception type.

```
try:
    age = int(input("Enter the age:"))
    if(age<18):
        raise ValueError
    else:
        print("the age is valid")
except Exception:
    print("The age is not valid")
print("outside exception")
```

output:

```
Enter the age:12
The age is not valid
outside exception
```

Raise the exception with message

```
try:
    num = int(input("Enter a positive integer: "))
    if(num <= 0):
        # we can pass the message in the raise statement
        raise ValueError("That is a negative number!")
except Exception as e:
    print(e)
```

output:

```
Enter a positive integer: -2
That is a negative number!
```

```
try:
    var1 = int(input("Enter var1:"))
    var2 = int(input("Enter var2:"))
    if var2 == 0:
        raise ArithmeticError("Div with zero...")
    else:
        print("var1/var2 = ",var1/var2)
except ArithmeticError as e:
    print("The value of var2 can't be 0")
    print("Error:",e)
```

output:

Enter var1:10

Enter var2:0

The value of var2 can't be 0

Error: Div with zero...



Smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215